

Structures and Type Classes

Lecture 22

Section 1

Part I: Structures

Motivation

Throughout Lectures 18–21 we built everything from inductive declarations. But many of our types had a single constructor bundling several fields: the `DSigma` type in Lecture 20 bundled a natural number k with a vector of length k — a dependent pair of data; the weighted trees in Lecture 21 were pairs of a weight and a tree.

Single-constructor inductive types are so common that Lean provides dedicated syntax for them: the `structure` command. It generates the type, its constructor, and **projection functions** for each field — all automatically.

Understanding structures is a prerequisite for type classes, because in Lean, a **class is literally a structure** with one extra feature: automatic instance resolution.

Declaring a structure

```
structure Point ( $\alpha$  : Type) where  
  x :  $\alpha$   
  y :  $\alpha$ 
```

This declares a type `Point  $\alpha$` with a single constructor `Point.mk` and two projection functions `Point.x` and `Point.y`:

```
#check @Point.mk      -- { $\alpha$  : Type}  $\rightarrow \alpha \rightarrow \alpha \rightarrow \text{Point } \alpha$   
#check @Point.x       -- { $\alpha$  : Type}  $\rightarrow \text{Point } \alpha \rightarrow \alpha$   
#check @Point.y       -- { $\alpha$  : Type}  $\rightarrow \text{Point } \alpha \rightarrow \alpha$ 
```

What structure generates

We can ask Lean to show us what was generated:

```
#print Point
-- inductive Point.{u} : Type u → Type u
-- constructors:
-- Point.mk : {α : Type u} → α → α → Point α
```

The structure command produced an inductive type with one constructor, plus projections for each field. This is what we would have written by hand:

```
inductive Point' (α : Type) where
  | mk : α → α → Point' α
def Point'.x : Point' α → α | .mk a _ => a
def Point'.y : Point' α → α | .mk _ b => b
```

The structure command saves this boilerplate and adds further conveniences (record syntax, dot notation, with, extends).

Building structures

There are several ways to construct a value:

```
-- Using the constructor directly
def p1 := Point.mk 10 20

-- Using anonymous constructor syntax
def p2 : Point Nat := ⟨10, 20⟩

-- Using named field syntax (order does not matter)
def p3 : Point Nat := { x := 10, y := 20 }
def p4 : Point Nat := { y := 20, x := 10 }
```

The named field syntax $\{x := 10, y := 20\}$ is the most readable. The anonymous constructor $\langle 10, 20 \rangle$ is terser — we used it in Lecture 20 when writing $\langle k, ys \rangle$ for sigma types.

Dot notation

Given `p : Point Nat`, you can write `p.x` instead of `Point.x p`:

```
def p : Point Nat := { x := 10, y := 20 }  
#eval p.x          -- 10  
#eval p.y          -- 20
```

Dot notation extends beyond projections. If you define a function in the `Point` namespace, you can call it with dot notation:

```
def Point.add (p q : Point Nat) : Point Nat :=  
  { x := p.x + q.x, y := p.y + q.y }  
  
def q : Point Nat := { x := 3, y := 4 }  
#eval p.add q      -- { x := 13, y := 24 }
```

Lean sees `p.add q`, looks up `Point.add`, and inserts `p` at the first argument of type `Point`.

Record update

To create a new record that differs from an old one in just a few fields, use the **with** syntax:

```
def p : Point Nat := { x := 1, y := 2 }  
  
#eval { p with y := 3 }      -- { x := 1, y := 3 }  
#eval { p with x := 4 }      -- { x := 4, y := 2 }
```

Fields you do not mention are taken from the record after **with**. This is especially useful for structures with many fields.

Structure inheritance

Structures can **extend** other structures, inheriting their fields:

```
inductive Color where | red | green | blue
```

```
structure ColorPoint ( $\alpha$  : Type) extends Point  $\alpha$  where  
  c : Color
```

ColorPoint α has three fields: x and y (inherited from Point) and c (new).
Lean generates a coercion ColorPoint.toPoint that extracts the parent part.

```
def cp : ColorPoint Nat :=  
  { x := 1, y := 2, c := .red }  
#eval cp.x           -- 1 (inherited field)  
#eval cp.toPoint      -- { x := 1, y := 2 }
```

Building children from parents

You can build a child from a parent using with:

```
def cp2 : ColorPoint Nat :=  
  { p with c := .blue }    -- inherits x=1, y=2
```

Lean also supports **multiple inheritance**: a structure can extend several parents at once, merging their fields:

```
structure RGBValue where  
  red : Nat; green : Nat; blue : Nat  
  
structure RedGreenPoint (α : Type)  
  extends Point α, RGBValue where  
  no_blue : blue = 0
```

Structures vs. inductive types

Feature	Inductive	Structure
Constructors	any number	exactly one
Projections	write by hand	auto-generated
Dot notation	manual namespace trick	automatic
Record syntax	no	{field := val}
Record update	no	{r with ...}
Inheritance	no	extends

A structure is not a new concept in the type theory — the kernel sees a single-constructor inductive type. The structure command is **elaboration convenience**.

Rule of thumb: one constructor $\Rightarrow$ structure. Multiple constructors (Bool, Nat, List, HTree) $\Rightarrow$ inductive.

Section 2

Part II: From Structures to Classes

The problem: ad-hoc polymorphism

We want to write a function that works for “any type that supports addition.” The idea, due to Wadler and Blott (1989), is called **ad-hoc polymorphism**:

Parametric polymorphism: $\text{id} : \alpha \rightarrow \alpha$ works for every α , doing the same thing. No constraints.

Ad-hoc polymorphism: `add` works for many types (`Nat`, `Int`, `Float`), but does something different for each. It requires the type to support a specific operation.

A first attempt: pass the implementation

Package the addition operation in a structure and pass it explicitly:

```
structure Add' ( $\alpha$  : Type) where
  add :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 

def double' (s : Add'  $\alpha$ ) (x :  $\alpha$ ) :  $\alpha$  :=
  s.add x x

#eval double' { add := Nat.add } 10      -- 20
#eval double' { add := Int.add } 10      -- 20
```

This works, but every call site must supply the record manually. What if Lean could fill in the Add' argument *automatically*?

The key idea: `class` = `structure` + `resolution`

Replace `structure` with `class`:

```
class Add ( $\alpha$  : Type) where  
  add :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 
```

The declaration looks the same. The difference shows up when you *use* it. Compare the two versions of `double`:

```
-- with structure: caller must pass the record  
def double' (s : Add'  $\alpha$ ) (x :  $\alpha$ ) :  $\alpha$  := s.add x x  
  
-- with class: Lean finds the record automatically  
def double [Add  $\alpha$ ] (x :  $\alpha$ ) :  $\alpha$  := Add.add x x
```

The square brackets `[Add  $\alpha$ ]` mean: “this function needs an `Add` instance for α , but the caller does not provide it — Lean looks it up.”

Three kinds of arguments

Syntax	Name	Who provides it
<code>(x : A)</code>	explicit	the caller writes it
<code>{x : A}</code>	implicit	Lean infers it by unification
<code>[x : A]</code>	instance-implicit	Lean searches the instance database

With `structure`, you wrote `double' { add := Nat.add } 10` — passing the record by hand. With `class`, you write `double 10` — Lean finds the `Add Nat` instance automatically.

Declaring instances

An **instance** registers a value of a class for a particular type:

```
instance : Add Nat where  
  add := Nat.add
```

```
instance : Add Int where  
  add := Int.add
```

```
instance : Add Float where  
  add := Float.add
```

Each instance is added to a global database. Compare with the manual approach: $\{\text{add} := \text{Nat.add}\}$ is the *same* record. The difference is that we register it once and Lean inserts it everywhere.

Using instance-implicit arguments

Now we write `double` with square brackets:

```
def double [Add  $\alpha$ ] (x :  $\alpha$ ) :  $\alpha$  :=  
  Add.add x x  
  
#eval double 10           -- 20  
#eval double (10 : Int)   -- 20  
#eval double (7 : Float)  -- 14.0
```

When Lean encounters `double 10`, it determines $\alpha = \text{Nat}$, sees the constraint `[Add Nat]`, searches the instance database, and finds our instance. The instance is inserted as an invisible argument — you never write it, but the elaborator fills it in before the kernel checks the term.

Chaining instances

The real power: an instance can itself require other instances.

```
instance [Add  $\alpha$ ] [Add  $\beta$ ] : Add ( $\alpha \times \beta$ ) where  
  add p q := (Add.add p.1 q.1, Add.add p.2 q.2)  
  
#eval double (3, 5)           -- (6, 10)  
#eval double ((1, 2), 3)      -- ((2, 4), 6)
```

For `double ((1, 2), 3)`, the elaborator must find `Add ((Nat  $\times$  Nat)  $\times$  Nat)`. It chains: this requires `Add (Nat  $\times$  Nat)` and `Add Nat`; the first of those in turn requires two copies of `Add Nat`. The search is recursive, like Prolog with backtracking.

Section 3

Part III: Key Library Classes

How notation works

The notation $a + b$ is not built into the language. It desugars to a type class method call. The library defines a class `HAdd` (heterogeneous addition), and the notation is:

```
infixl:65 " + " => HAdd.hAdd
```

When you write $a + b$, Lean elaborates it as `HAdd.hAdd a b` and resolves the instance by type class search. The same pattern applies to all standard operators:

$$\begin{array}{llll} a + b & \rightarrow \text{HAdd.hAdd } a \ b & a \leq b & \rightarrow \text{LE.le } a \ b \\ a * b & \rightarrow \text{HMul.hMul } a \ b & a == b & \rightarrow \text{BEq.beq } a \ b \end{array}$$

To make your type support $+$, just provide an `Add` instance. A default instance in the library automatically derives the `HAdd` instance from it, and the notation works.

OfNat: polymorphic numeric literals

In Lean, $(2 : \text{Nat})$, $(2 : \text{Int})$, and $(2 : \text{Float})$ all typecheck. The literal 2 is **polymorphic** — Lean decides what it means via a type class:

```
class OfNat ( $\alpha$  : Type) (n : Nat) where  
  ofNat :  $\alpha$ 
```

When Lean sees the numeral 2 at type α , it searches for an instance of `OfNat  $\alpha$  2`. Different types provide different instances: `Nat` maps 2 to `succ (succ zero)`, `Int` maps it to the integer 2, and so on. If no instance exists, the literal does not typecheck.

OfNat: example

```
structure Rational where
  num : Int; den : Nat; inv : den ≠ 0

instance : OfNat Rational n where
  ofNat := { num := n, den := 1, inv := by decide }

instance : ToString Rational where
  toString r := s! "{r.num}/{r.den}"

#eval (2 : Rational)      -- 2/1
#check (2 : Nat)         -- 2 : Nat
```

The Rational structure bundles three fields (a numerator, denominator, and an invariant proof). The OfNat instance constructs a rational from any numeral by placing it over 1.

Inhabited: default elements

```
class Inhabited ( $\alpha$  : Type) where
  default :  $\alpha$ 

instance : Inhabited Bool where default := true
instance : Inhabited Nat where default := 0

instance [Inhabited  $\alpha$ ] [Inhabited  $\beta$ ]
  : Inhabited ( $\alpha \times \beta$ ) where
    default := (Inhabited.default, Inhabited.default)

#eval (Inhabited.default : Nat  $\times$  Bool)    -- (0, true)
```

Inhabited is used whenever Lean needs a fallback value (e.g. `List.head!` on an empty list). It chains naturally: if α and β are inhabited, so is $\alpha \times \beta$.

ToString

ToString works similarly. Implement it for your type, and chaining handles compound types:

```
structure Person where
  name : String
  age  : Nat

instance : ToString Person where
  toString p := p.name ++ "@" ++ toString p.age

#eval toString { name := "Leo", age := 542 : Person }
-- "Leo@542"
```

This uses several features at once: named-field syntax to build the Person, dot notation to access its fields, and instance resolution to find ToString String and ToString Nat for the recursive calls.

Section 4

Part IV: Decidable Propositions

The Decidable class

In Lecture 21 we wrote `if n = s then ... else ...` and noted that this requires Lean to *decide* whether $n = s$ at runtime. The mechanism is a type class:

```
class inductive Decidable (p : Prop) where  
  | isFalse (h : ¬p) : Decidable p  
  | isTrue   (h : p)  : Decidable p
```

A natural question: we said in Lecture 18 that Prop has proof irrelevance — all proofs of a proposition are equal. So how can we pattern-match on `isTrue` vs. `isFalse`?

The answer: `Decidable p` lives in `Type`, not `Prop`. The *proposition* p is a `Prop`, but the *decision* is data. The constructor tag (`isTrue` or `isFalse`) is computationally relevant; only the proof h inside is erased.

How if works

The expression `if c then a else b` desugars to `ite c a b`:

```
def ite (c : Prop) [h : Decidable c]
  (t e :  $\alpha$ ) :  $\alpha$  :=
  match h with
  | .isTrue   _ => t
  | .isFalse  _ => e
```

If Lean cannot find a `Decidable` instance for `c`, the `if` does not compile. There is also `dite` (dependent if-then-else) with syntax `if h : c then ... else ...`, where `h` binds the proof in each branch.

DecidableEq and deriving

`DecidableEq  $\alpha$` is shorthand for $\forall (a\ b : \alpha), \text{Decidable } (a = b)$. In Lecture 21:

```
inductive Bit : Type where  
  | 0 : Bit  
  | 1 : Bit  
deriving DecidableEq
```

The deriving clause auto-generates an instance that pattern-matches on both arguments, returning `isTrue rfl` when they match and `isFalse ...` when they do not. This is why `split at h` in Lecture 21 gave us a proof of $n = s$ or $n \neq s$.

Decidability composes

The standard library provides instances showing decidability is preserved under connectives:

```
-- if p and q are decidable, so are  $p \wedge q$ ,  $p \vee q$ , and  $\neg p$   
instance [Decidable p] [Decidable q]  
  : Decidable (p  $\wedge$  q) := ...  
instance [Decidable p] [Decidable q]  
  : Decidable (p  $\vee$  q) := ...  
instance [Decidable p] : Decidable ( $\neg$ p) := ...
```

This is instance chaining again. To compile `if x < a  $\vee$  x > b then 0 else 1`, the elaborator needs `Decidable (x < a  $\vee$  x > b)`. It chains: the $\vee$ instance requires `Decidable (x < a)` and `Decidable (x > b)`, both provided by the library's instance for `<` on `Nat`. A compound decision procedure, assembled automatically.

The decide tactic

Because Decidable carries proofs, it doubles as a proof tactic:

```
example : 10 < 5 ∨ 1 > 0 := by decide
example : ¬(True ∧ False) := by decide
example : 10 * 20 = 200    := by decide
```

This works for **closed** propositions (no free variables) where the computation terminates in reasonable time. It will not work for universally quantified statements like $\forall n, n + 0 = n$ — those need induction, not case enumeration.

Section 5

Part V: Coercions and Scope

Coercions via type classes

Lean implements coercions as type class instances:

```
instance : Coe Nat Int where  
  coe := Int.ofNat
```

When Lean expects Int but receives a Nat , it searches for `Coe Nat Int` and inserts the coercion automatically. Lean chains coercions transitively: if there is a coercion from A to B and from B to C , Lean can coerce from A to C .

Local and scoped instances

Instances are global by default. For limited scope:

```
section  
local instance : Add (Point Nat) where  
  add a b := { x := a.x + b.x, y := a.y + b.y }  
def doublePoint (p : Point Nat) := p + p  
end   -- Add (Point Nat) no longer active
```

Scoped instances activate when their namespace is opened:

```
namespace Point  
scoped instance : Add (Point Nat) where  
  add a b := { x := a.x + b.x, y := a.y + b.y }  
end Point   -- not active outside
```

This is important in Mathlib, where scoped instances manage interactions between thousands of instances.

Section 6

Part VI: Discussion

A complete example: the type

```
inductive Parity : Type where
```

```
| even : Parity
```

```
| odd  : Parity
```

```
instance : Add Parity where
```

```
add
```

```
| .even, p      => p
```

```
| .odd,  .even => .odd
```

```
| .odd,  .odd  => .even
```

```
instance : ToString Parity where
```

```
toString | .even => "even" | .odd => "odd"
```

A complete example: more instances

```
instance : OfNat Parity 0 where ofNat := .even
instance : OfNat Parity 1 where ofNat := .odd

instance : BEq Parity where
  beq a b := match a, b with
    | .even, .even | .odd, .odd => true
    | _, _ => false

#eval double (1 : Parity)          -- even
#eval (0 : Parity) + (1 : Parity)  -- odd
```

Five short instance declarations give us `+`, `==`, numeric literals, and string output. Each instance is a few lines; the type class machinery does the rest.

What the kernel sees

Type classes are not primitive in CIC. They are **elaboration sugar**:

Source	Kernel term
<code>structure Point</code>	single-constructor inductive type
<code>class Add</code>	the same (single-constructor inductive)
<code>instance : Add Nat</code>	a value of that type, registered
<code>[Add α]</code>	explicit argument, filled by search

The kernel knows nothing about type classes. It sees inductive types, constructors, and eliminators. The `structure` and `class` commands, `dot` notation, instance resolution — all are elaboration-time mechanisms.

What we have now explained

Usage in L19–21	Mechanism	What it does
<code>if n = s then ...</code>	<code>DecidableEq</code> (class)	decides equality
deriving <code>DecidableEq</code>	auto instance	builds decision proc
$a + b$ on <code>Nat</code>	<code>Add</code> / <code>HAdd</code>	overloaded notation
$a \leq b$ on <code>Nat</code>	<code>LE</code>	overloaded notation
<code>0</code> as a numeral	<code>OfNat</code>	polymorphic literal
<code>#eval</code> output	<code>Repr</code>	display formatting
<code>.x</code> , <code>.y</code> access	projections	auto-generated
<code>{x := ...}</code>	record syntax	structure-specific
<code>⟨k, ys⟩</code>	anon. constructor	single-ctor types

Exercises (1)

- 1 Define a structure `Complex` with fields `re : Float` and `im : Float`. Provide instances for `Add`, `Mul`, `ToString`, and `OfNat` (mapping n to $n + 0i$). Verify with `#eval` that $(1 + 2i)(3 + 4i) = -5 + 10i$.
- 2 Define a structure `ColorPoint` extending `Point Nat` with a `Color` field. Build a value, modify it with record update, and access both inherited and new fields.
- 3 Define a class `Monoid ( $\alpha$  : Type)` with fields `unit :  $\alpha$` and `op :  $\alpha \rightarrow \alpha \rightarrow \alpha$` . Provide instances for `Nat` (with `0` and `+`) and for `List  $\alpha$` (with `[]` and `append`). Write a function `fold : [Monoid  $\alpha$ ]  $\rightarrow$  List  $\alpha \rightarrow \alpha$` that folds a list using the monoidal operation.

Exercises (2)

- 4 Implement DecidableEq for Parity by hand (no deriving). In each branch, produce isTrue rfl or isFalse (fun h $\Rightarrow$ Parity.noConfusion h). Why does noConfusion work?
- 5 (Harder) Define a class Finite (α : Type) with a field elems : List α and a proof complete : $\forall (x : \alpha), x \in \text{elems}$. Provide instances for Bool and Parity. Write forAll : [Finite α] $\rightarrow (\alpha \rightarrow \text{Bool}) \rightarrow \text{Bool}$.
- 6 (Open-ended) In Lecture 20 we defined BST *lo hi* using hard-coded Nat comparisons. Refactor it using an Ord type class so the BST works for any ordered type. What changes?

All exercise stubs are in the companion Lean file.

What we covered

- 1 **Structures:** the structure command, projections, dot notation, record syntax, record update, and inheritance with extends.
- 2 **From structures to classes:** a class is a structure with instance-implicit arguments and automatic resolution.
- 3 **Instances and chaining:** the instance database, Prolog-like backward search, recursive resolution.
- 4 **Key library classes:** Add/HAdd, OfNat, Inhabited, ToString, BEq, LE.
- 5 **Decidable propositions:** Decidable, DecidableEq, how if works, decide, and deriving.
- 6 **Coercions:** Coe as a type class for implicit conversions.

Next lecture: well-founded recursion and proof automation.

How to write recursive functions that do not recur on a direct subterm. The `WellFoundedRelation` class, `termination_by`, `decreasing_by`, and the `Acc` type. Then: how `simp`, `omega`, and `decide` automate the proofs we have been writing by hand, and how to register your own `@[simp]` lemmas.